

杂题选讲

251Sec

2026/5/14

有 $N = 2^n$ 名选手，编号为 1 到 N ，他们被排列在高度为 n 的满二叉树的叶节点上。每个内部节点处，两个子树的胜者对决，编号较小者获胜进入下一轮，最终根节点产生冠军。定义“木勺奖”得主为满足以下条件的选手：他在第一轮落败；击败他的选手在第二轮落败；击败那位选手的选手在第三轮落败；……；依此类推，直到决赛中的败者。对于每个选手，求在所有 $(2^n)!$ 种排列中，他获得木勺奖的方案数，对 998244353 取模后输出。

$1 \leq n \leq 20$ 。

不妨规定考虑的那个人在第一个位置，最后答案乘 2^n 即可。

原问题等价于：把序列分成长度为 $1, 1, 2, 4, 8, \dots, 2^{n-1}$ 的段，要求每个段最小值依次递减，且第一个段的值是给定值。

考虑从小到大插入所有数，并维护当前已经有了数的所有段（显然是一个后缀）。

设 $f(i, j)$ 代表已经插入了 $1 \sim i$ ，目前已经插入了 j 段。转移就是开一个新段或者插入已有的段。算答案是简单的。

复杂度 $O(n2^n)$ 。

给定数列 $\{a_n\}$ ，求一个字典序最大的子序列，使得这个子序列可以成为多边形的边长。
一个子序列能成为多边形的边长当且仅当它的长度至少为 3，且最大值的 2 倍小于所有值的和。

$3 \leq n \leq 2 \times 10^5$, $1 \leq a_i \leq 10^9$ 。

考虑一下，假如确定了最大值要怎么办。那么限制就是所有数的和大于一个值。直接每次贪心取一个尽可能长的前缀，使得最终必须要在在这个前缀里取一个元素，然后从这个前缀选取一个最大的元素即可。枚举最大值可以做到 $O(n^2 \log n)$ 。

我们注意到：如果原序列的最大值不被选，那么所有大于最大值一半的必定不会选，因为否则选上最大值会更优。因此，可能成为最大值的数只有 $O(\log V)$ 种，这就做到了 $O(n \log n \log V)$ 。

给定一个字符集为 ATCG 的字符串。对于一个正整数 k ，显然共有 4^k 种长度为 k 的字符串。如果原字符串包含所有这 4^k 种字符串作为子序列，则它是 k -好的。你需要对所有 $1 \leq k \leq n$ ，求最少在原字符串修改多少个字符，能够使得它变成 k -好的，或者判断无解。

$1 \leq n \leq 2 \times 10^5$ 。

原问题可以被转换成：把原序列划分成 k 个长度至少为 4 的段，每段的价值是颜色数，求最大的价值和。
可以发现，价值满足四边形不等式，因此 WQS 二分可以求解单组询问。

本题的核心性质在于答案值域 $\leq n$ 。设 WQS 二分的斜率为 c 。若 $c > \sqrt{n}$ ，则必定有 $k \leq \sqrt{n}$ 。
因此，只需要对所有的 $c, k \leq \sqrt{n}$ 计算 DP 值。这就做到了 $O(n\sqrt{n})$ 。

有没有更好写的做法呢。WQS 二分实际上是在切一个凸壳，而我们要做的事情是求出整个凸壳。

本题的凸壳的重要性质是：它只有 $O(\sqrt{n})$ 个拐点。考虑对凸壳分治：假设我们要求出 $[L, R]$ 内的所有拐点，只需用 L, R 连线的斜率尝试切凸壳，如果切到了一个拐点 M ，则递归 $[L, M], [M, R]$ ，否则说明 $[L, R]$ 内没有拐点，可以直接结束。这样的复杂度也是 $O(n\sqrt{n})$ 的。

考虑如下问题：

- 二维平面上有 n 个点 (x_i, y_i) 。
- 你需要计算有多少满足以下条件的点对 (i, j) ：
 - (i, j) 的切比雪夫距离小于等于所有 (i, k) ($1 \leq k \leq n$) 的切比雪夫距离。
 - (i, j) 的曼哈顿距离大于等于所有 (i, k) ($1 \leq k \leq n$) 的曼哈顿距离。

你需要从一个空的平面开始进行 n 次加点，每次加点后回答这个问题的答案。
 $2 \leq n \leq 4 \times 10^5$ 。

先做静态问题。设所有点对的最大曼哈顿距离为 L ，则对于一个点 a ，和它曼哈顿距离最远的点的距离至少是 $\frac{L}{2}$ ，于是它们的切比雪夫距离至少是 $\frac{L}{4}$ 。

把平面分块成 $\frac{L}{4} \times \frac{L}{4}$ 的正方形，如果一个正方形内有至少 2 个点，那么它们一定都不会成为合法的点对 (a, b) 中的 a 。正方形个数是 $O(1)$ 的，于是我们得到了 $O(1)$ 个可能成为 a 的点，接下来暴力做就好了。

然后考虑一下加点怎么办呢，只需要在 L 翻倍的时候重构网格即可。复杂度是 $O(n \log V)$ 。

给定 n, k , 求有多少 n 个点的有标号无向图, 满足最小点覆盖大小为 k 。对 2 取模。多组询问, 每次给定 n, k 。

$1 \leq T \leq 2^{14}, 1 \leq n < 2^{14}, 0 \leq k < n$ 。

先转成最大独立集。

考虑点 1, 2。如果它们的邻域不同，那么交换它们的标号就抵消了。因此它们邻域必相同。如果它们之间有边，那么最多选一个，可以删去任意一个；如果它们之间没有边，那么要么都选、要么都不选，可以合并成大小为 2 的点。

这个合并过程是可以重复的，任意两个大小相同的点，都可以进行合并。

我们如下规定合并的过程：从 $1 \sim n$ 依次加入图中的点，每次加入后，找到两个大小相同的点进行合并，重复直到所有点大小不同。显然，每个时刻可以合并的点只有至多一对，因此这个过程是良定义的。并且上述抵消在这个过程中可以验证是双射。最终，图里会有一些大小为 $2^{a_1}, 2^{a_2}, \dots, 2^{a_k}$ 的点，满足 $a_1 > a_2 > \dots > a_k$ 。这个图的最大独立集是容易计算的：只需要从 a_1 到 a_k 贪心选取即可。

接下来，我们开始对这个过程计数。首先设 $f(n, m)$ 代表 n 个点合并完成后总大小为 m ，且尚未考虑最终的 $\text{popc}(m)$ 个点之间的连边如何构成的方案数。 $g(m)$ 为对于一张总大小为 m 的图，合法的连边的方案数。

先计算 $f(n, m)$ ：考虑加入最后一个点的变化过程，可得

$$f(n, m) = f(n-1, m-1) + f(n-1, m-1 + \text{lowbit}(m))。$$

然后计算 $g(m)$ ：实际上，如果有一个 $a_1, a_2, \dots, a_{k-1}$ 中的点没被选，那么 a_k 到它的连边是否存在是无所谓的，这就抵消了。因此， $a_1, \dots, a_{k-1}$ 一定全都在独立集里，只需计算 a_k 在与不在的贡献即可。

有一棵 n 个点的未知的树，其中每条边都被定向了。你可以进行 $Q = 50$ 次询问。每次询问你翻转任意一个边的子集的方向，交互库会告诉你此时每个点 u 能被多少个点到达，记为 f_u 。询问结束后边的方向会被翻转回来。
你需要还原整棵树和初始时边的定向。 $2 \leq n \leq 10^4$ 。

树形态相关的交互题，一个经典的想法是剥叶子。这个题的叶子有一个很强的性质：如果它的出边被向外定向，则它的答案是 1。

那么一个大致的思路就是：随机所有边的方向询问，那么 “ $f_u = 1$ 在回答里大致占了一半的点” 就是这叶子。

我们尝试确定化这个策略，这就是经典 trick 了：对于每条边，随机一个包含 $\frac{Q}{2}$ 个 1 的二进制数，使得所有边的这个二进制数互不相等且互不为补集。对于第 i 次询问，将所有第 i 位为 1 的边翻转。

此时，如果一个点是叶子，那么它的邻边一定恰好向外指 $\frac{Q}{2}$ 次。而如果一个点不是叶子，则它的邻边全部向外的次数一定小于 $\frac{Q}{2}$ 。

这样，我们就找到了一个叶子，接下来我们需要确定和它相连的边的方向和它的父亲。确定边的方向是简单的：只需要找到一条边的二进制数和这个点 $f_u = 1$ 的询问相等或互补即可。而对于它的父亲 v ，我们发现这样一个性质：一旦 $f_u > 1$ ，则 $f_u = f_v + 1$ 。我们猜想：满足这个条件的点极大概率是唯一的，它就是 u 的父亲。

我们考虑这个点什么时候不唯一：假设我们错判成了一个点 w ，那么在这次询问里 $f_v = f_w$ ，然而，由于 v, w 的出边方向是随机的，那么只要任意翻转一条出边，就会出现 $f_v \neq f_w$ ，从而单次判断的错误率是 $\leq 2^{-Q/2}$ 级别的。

给定一张无向图，其中有一些边是特殊边。求一个简单环，使得所有特殊边要么在环上，要么和环上的点无交。若不存在，报告无解。

$2 \leq n \leq 150$, $1 \leq m \leq \binom{n}{2}$ 。

首先做一些简单的处理：

- 如果一个点相连的关键边数量至少为 3，则这个点可以被删除。
- 如果一个点相连的关键边数量为 2，则这个点的所有非关键边邻边可以被删除。
- 如果一条边是割边，它可以被删除。
- 如果删除了一条关键边，则与它相连的点应该被删除。

我们宣称，如果重复进行上述操作之后，图中有剩余的边，则一定有解。
首先，如果存在关键边组成的环那直接就搞定了。否则，所有的关键边形成若干条链，链除了端点以外的点一定不存在非关键边相连，因此所有这样的链可以被缩成一条边。

我们考虑此时的图的形态（不妨只考虑一个非空的连通块）：

- 它是边双连通的。
- 所有的关键边两两没有交点。

接下来，考虑依次加入每条关键边，并维护一个环（初始时任找一个环即可，因为图是边双所以一定能找到）。

- 若加入的关键边与环无交，则无视它。
- 若加入的关键边与环有两个交点，则说明在环上和它有交点的边都一定不是关键边，直接把一整段弧删掉换成它即可。
- 若加入的关键边与环有一个交点，考虑一张新图，在这张新图上将环缩成一个点 p ，此时，上述两个关于图形态的陈述仍然满足，因此可以在新图里找到一个环。若它不经过 p 那直接就对了，否则只需要在 p 的这个环上找到一条连接两端的路径就行了。

这已经给出了有解性的证明，并且这也已经是一个可以实现的算法了。

但是有没有更简洁的方法呢。有的有的，我们直接依次枚举每条边，判断删除它之后是否有解，如果有解就删掉。那么最后就会剩下一个环，输出它即可。